Software Design Specifications

for

Digital Complaint & Grievance Management System

Prepared by: Team 49

- SE23UCSE118 - Nandini Penmetsa
- SE23UCSE129 - N. Nithin Reddy
- SE23UCSE130 - Amulya Oruganti
- SE23UCSE133 - P. Indira Sneha
- SE23UCSE152 - Hanshika Ravipati
- SE23UCSE156 - S. Vaishnavi Reddy
- SE23UCSE159 - S. Amulya
- SE23UCSE164 - Sreeja Sunkara
- SE23UCSE060 - Abhiram Reddy
- SE23UCSE062 - Thanishq Reddy

Document Information

Title:			
Project Manager:		**Document Version No:**	
		Document Version Date:	
Prepared By:		**Preparation Date:**	

Version History

Ver. No.	Ver. Date	Revised By	Description	Filename

Table of Contents

1 INTRODUCTION 4

1.1 PURPOSE 4
1.2 SCOPE 4
1.3 DEFINITIONS, ACRONYMS, AND ABBREVIATIONS 4
1.4 REFERENCES 4

2 USE CASE VIEW 4

2.1 USE CASE 4

3 DESIGN OVERVIEW 4

3.1 DESIGN GOALS AND CONSTRAINTS 5
3.2 DESIGN ASSUMPTIONS 5
3.3 SIGNIFICANT DESIGN PACKAGES 5
3.4 DEPENDENT EXTERNAL INTERFACES 5
3.5 IMPLEMENTED APPLICATION EXTERNAL INTERFACES 5

4 LOGICAL VIEW 5

4.1 DESIGN MODEL 6
4.2 USE CASE REALIZATION 6

5 DATA VIEW 6

5.1 DOMAIN MODEL 6
5.2 D ATA MODEL (PERSISTENT DATA VIEW) 6
5.2.1 Data Dictionary 6

6 EXCEPTION HANDLING 6

7 CONFIGURABLE PARAMETERS 6

8 QUALITY OF SERVICE 7

8.1 AVAILABILITY 7
8.2 SECURITY AND AUTHORIZATION 7
8.3 LOAD AND PERFORMANCE IMPLICATIONS 7
8.4 MONITORING AND CONTROL 7

1 Introduction

1.1 Purpose

The purpose of this Software Design Specification (SDS) document is to describe the design of the **Digital Complaint and Grievance System** based on the requirements specified in the Software Requirements Specification (SRS).This document provides detailed information about the system architecture, design components, module interactions, and data handling mechanisms. It acts as a reference for developers, testers, and stakeholders to understand how the system will be implemented.

The SDS also serves as a foundation for system development, testing, and future enhancements.

1.2 Scope

The Digital Complaint and Grievance System is a **web-based application** that allows users to submit complaints digitally and track their status, while administrators manage and resolve complaints.

The system includes the following major functionalities:

- User registration and authentication
- Complaint submission with supporting files
- Complaint tracking and history viewing
- Administrator dashboard for complaint management
- Filtering and updating complaint status

The system is implemented using:

- **Frontend:** React
- **Backend:** Spring Boot
- **Database:** MySQL

This document focuses on the **design aspects** of these functionalities, including system structure, data flow, and interaction between components.

1.3 Definitions, Acronyms, and Abbreviations

- **SRS** – Software Requirements Specification
- **SDS** – Software Design Specification
- **UI** – User Interface
- **API** – Application Programming Interface
- **Admin** – Administrator responsible for managing complaints
- **User** – Individual who submits complaints
- **DBMS** – Database Management System
- **Authentication** – Process of verifying user identity
- **Authorization** – Process of granting access permissions

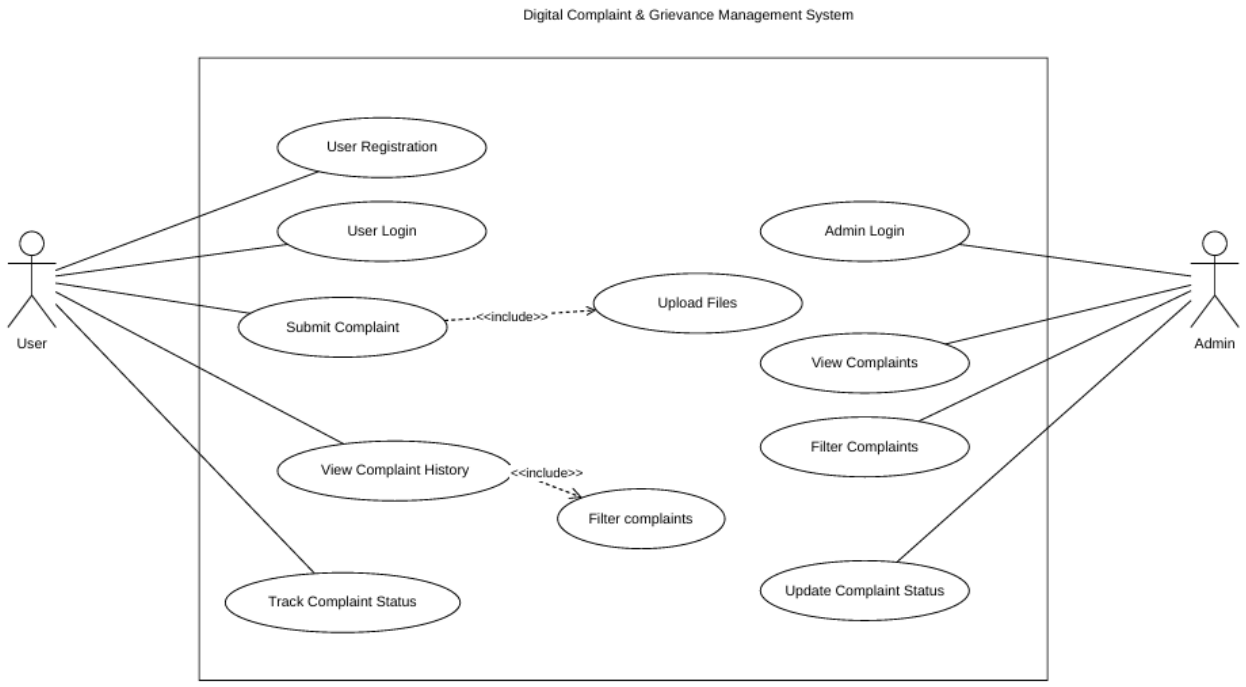
1.4 References

The following documents and resources were referred to in preparing this Software Design Specification:

- Software Requirements Specification (SRS) for Digital Complaint and Grievance System
- IEEE Recommended Practice for Software Design Documentation
- Software Engineering – Ian Sommerville
- UML documentation and modeling guidelines
- Course materials provided by the instructor

2 Use Case View

The Digital Complaint and Grievance Management System includes two main actors: **User** and **Admin**. The use case view presents the major functionalities of the system and shows how these actors interact with the system. The overall use case diagram below provides a high-level representation of the important system functions such as registration, login, complaint submission, complaint tracking, complaint history viewing, and administrative complaint management. These use cases are based on the requirements defined in the SRS.



2.1 Use Case

This section describes the major use cases of the Digital Complaint and Grievance Management System. These use cases represent the key functionalities of the system and define how users and administrators interact with the system.

2.1.1 User Registration

Brief Description:
This use case allows a new user to create an account in the system by providing required details such as name, email, and password.

Actor:
User

Preconditions:

- The user must have access to the registration page
- The user must not already have an account

Procedure:

1. The user opens the registration page
2. The system displays the registration form
3. The user enters required details
4. The system validates the entered data
5. The system creates a new user account

Postconditions:

- A new user account is created and stored in the database

2.1.2 User Login

Brief Description:
This use case allows a registered user to log into the system using valid credentials.

Actor:
User

Preconditions:

- The user must have a registered account

Procedure:

1. The user opens the login page
2. The system displays the login form
3. The user enters username/email and password
4. The user clicks the login button
5. The system validates the credentials

6. If valid, the system grants access to the user dashboard

Postconditions:

- The user is successfully logged into the system

2.1.3 Submit Complaint

Brief Description:
This use case allows a user to submit a complaint and optionally upload supporting files.

Actor:
User

Preconditions:

- The user must be logged into the system

Procedure:

1. The user opens the complaint submission page
2. The system displays the complaint form
3. The user enters complaint details
4. The user uploads supporting files (optional)
5. The user submits the complaint
6. The system validates and stores the complaint

Postconditions:

- The complaint is stored in the database
- A complaint ID is generated

2.1.4 Track Complaint Status

Brief Description:
This use case allows the user to track the current status of a submitted complaint.

Actor:
User

Preconditions:

- The user must be logged in
- The user must have submitted at least one complaint

Procedure:

1. The user opens the complaint tracking page
2. The system retrieves complaint details
3. The system displays the current status
4. The user views updates related to the complaint

Postconditions:

- The user can view the complaint status and updates

2.1.5 View Complaint History

Brief Description:
This use case allows the user to view previously submitted complaints.

Actor:
User

Preconditions:

- The user must be logged into the system

Procedure:

1. The user opens the complaint history page
2. The system retrieves past complaints
3. The system displays the list of complaints

Postconditions:

- The user can view all previously submitted complaints

2.1.6 Admin Login

Brief Description:
This use case allows the administrator to log into the system and access administrative functions.

Actor:
Admin

Preconditions:

- The administrator must have valid credentials

Procedure:

1. The administrator opens the login page
2. The system displays the login form
3. The administrator enters credentials
4. The system validates the credentials
5. The system grants access to the admin dashboard

Postconditions:

- The administrator is successfully logged into the system

2.1.7 View All Complaints

Brief Description:
This use case allows the administrator to view all complaints submitted in the system.

Actor:
Admin

Preconditions:

- The administrator must be logged into the system

Procedure:

1. The administrator opens the complaints section
2. The system retrieves all complaint records
3. The system displays all complaints

Postconditions:

- The administrator can view all complaints

2.1.8 Filter Complaints

Brief Description:
This use case allows the administrator to filter complaints based on specific criteria.

Actor:
Admin

Preconditions:

- The administrator must be logged in
- Complaint records must exist

Procedure:

1. The administrator selects filter criteria
2. The system applies the filter
3. The system displays matching complaints

Postconditions:

- Filtered complaints are displayed

2.1.9 Update Complaint Status

Brief Description:
This use case allows the administrator to update the status of a complaint.

Actor:
Admin

Preconditions:

- The administrator must be logged in
- Complaint must exist in the system

Procedure:

1. The administrator selects a complaint
2. The system displays complaint details
3. The administrator updates the complaint status
4. The system saves the updated status

Postconditions:

- The complaint status is updated and visible to users

3 Design Overview

The **Digital Complaint and Grievance System** is designed as a **web-based three-tier application** consisting of a presentation layer, an application layer, and a data layer. The system enables users to register, log in, submit complaints, upload supporting documents, track complaint status, and view complaint history. It also enables administrators to log in, view complaints, filter them, and update complaint status. This design follows the functional requirements defined in the SRS and supports modular development and future enhancement.

The high-level module decomposition of the system is as follows:

- **Presentation Layer:** React-based web interface for users and administrators
- **Application Layer:** Spring Boot backend exposing REST APIs and handling business logic
- **Data Layer:** MySQL database for storing users, complaints, status updates, and attachments metadata

The design emphasizes modularity, maintainability, security, and clear separation of responsibilities between frontend, backend, and database components.

3.1 Design Goals and Constraints

Design Goals

The main goals of the system design are:

- To provide a simple and user-friendly interface for complaint submission and tracking
- To enable administrators to efficiently manage and update complaints
- To ensure secure authentication and access control for both users and administrators
- To maintain complaint data reliably in a centralized database
- To support modular development so that future enhancements can be added easily
- To provide acceptable performance for multiple simultaneous users, with normal response time within three seconds as mentioned in the SRS

Design Constraints

The following constraints have significant impact on the software design:

- The frontend must be implemented using **React**
- The backend must be implemented using **Spring Boot**
- The database must be implemented using **MySQL**
- The application must run through standard web browsers
- The system must follow basic web security practices and restrict admin functions to authorized administrators only
- The design should remain suitable for academic project development within the available team schedule and scope
- No legacy system integration is assumed in the current version

Design Strategy

The system follows a layered architecture so that user interface logic, business logic, and database operations remain separated. This makes testing, debugging, and maintenance easier. REST APIs are used for communication between the React frontend and Spring Boot backend.

3.2 Design Assumptions

The software design is based on the following assumptions:

- Users and administrators have access to devices with internet connectivity
- The system is accessed through a modern web browser
- The MySQL database server is available during system operation
- The backend REST APIs are available for frontend communication

- Users must register before logging in and submitting complaints
- Administrators already have valid admin credentials created in the system
- Complaint data, uploaded files, and status updates can be stored and retrieved successfully from the database
- The number of users and complaints will remain within the manageable scope of the project deployment environment
- Uploaded files will be limited by supported file type and size, as described in the use cases

3.3 Significant Design Packages

The system is decomposed into the following significant design packages/modules:

1. Presentation Package

This package contains the React user interface components and pages.

Main responsibilities:

- User registration page
- User login page
- Complaint submission form
- Complaint tracking page
- Complaint history page
- Admin login page
- Admin dashboard for viewing, filtering, and updating complaints

2. Controller Package

This package contains backend controllers developed using Spring Boot.

Main responsibilities:

- Accept HTTP requests from the frontend
- Route requests to appropriate services
- Return responses to the client through REST APIs

Examples:

- User controller
- Auth controller
- Complaint controller
- Admin controller

3. Service Package

This package contains the business logic of the application.

Main responsibilities:

- Validate user input

- Authenticate users and administrators
- Handle complaint submission and retrieval
- Process complaint status updates
- Manage complaint filtering and tracking logic

4. Repository Package

This package handles communication with the MySQL database.

Main responsibilities:

- Store and retrieve user data
- Store and retrieve complaint records
- Store complaint status updates
- Access file metadata or linked attachment records

5. Model / Entity Package

This package contains domain classes representing persistent system data.

Examples:

- User
- Admin
- Complaint
- Attachment
- StatusUpdate / Response

6. Database Package

This package represents the MySQL schema and persistent storage structures required by the system.

This package decomposition supports separation of concerns and allows each layer to be developed and tested independently. It also matches the three-tier architecture described in the SRS.

3.4 Dependent External Interfaces.

External Application and Interface Name	Module Using the Interface	Functionality / Description
Web Browser Interface	Presentation Layer	Used by users and administrators to access the React-based web application
React Frontend	User Interface Module	Provides forms, dashboards, and pages for interacting with the system
Spring Boot REST API	Frontend and Backend Integration Module	Used by the frontend to send requests such as login, registration, complaint submission, tracking, and admin operations

MySQL Database Interface	Repository / Data Access Module	Used by backend modules to store and retrieve users, complaints, status, and file-related data
File Upload Interface	Complaint Submission Module	Used to upload supporting documents while submitting a complaint

3.5 Implemented Application External Interfaces (and SOA web services)

Interface Name	**Module Implementing the Interface**	**Functionality / Description**
User Registration API	Auth / User Controller	Accepts new user details, validates input, and creates a user account
User Login API	Auth Controller	Authenticates user credentials and grants access
Complaint Submission API	Complaint Controller	Accepts complaint details and optional supporting file data
Complaint History API	Complaint Controller	Retrieves complaints submitted by the logged-in user
Complaint Tracking API	Complaint Controller	Returns complaint status and related updates
Admin Login API	Admin Controller	Authenticates administrator credentials
View All Complaints API	Admin / Complaint Controller	Retrieves all complaints for admin review
Filter Complaints API	Admin / Complaint Controller	Returns complaints matching selected criteria such as status or date
Update Complaint Status API	Admin / Complaint Controller	Allows administrator to update complaint status

4 Logical View

The Logical View describes the internal structure of the Digital Complaint and Grievance Management System. It focuses on the main software components, classes, and their relationships required to implement the system functionality. This view explains the design of the system and how the modules interact to attain the desired behaviour. It also shows how use cases are realized through interaction of system components.

4.1 Design Model

The system follows a layered architecture consisting of multiple interacting classes grouped into modules.

- **Main Classes**

1. User Class

- Attributes: userId, name, email, password
- Methods: register(), login()

2. Admin Class

- Attributes: adminId, username, password
- Methods: login(), viewComplaints(), updateStatus()

3. Complaint Class

- Attributes: complaintId, title, description, status, date
- Methods: submitComplaint(), updateStatus()

4. Attachment Class

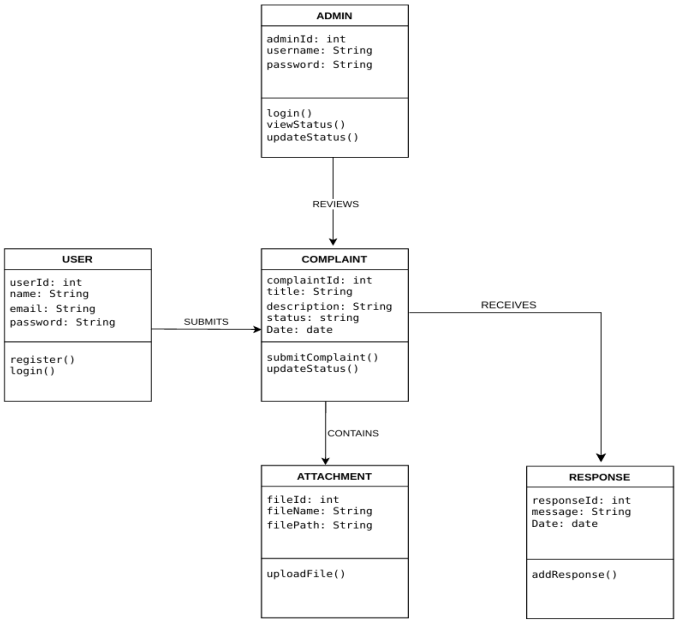
- Attributes: fileId, fileName, filePath
- Methods: uploadFile()

5. Response Class

- Attributes: responseId, message, date
- Methods: addResponse()

- **Relationships**

- User → submits → Complaint
- Complaint → has → Attachment
- Admin → manages → Complaint
- Complaint → contains → Response



4.2 Use Case Realization

This section describes how each use case is realized within the system design. It explains how different layers of the system such as the Presentation Layer, Controller Layer, Service Layer, and Repository Layer interact to fulfill the required functionality.

U1 – User Registration

Actors: User

Description:
This use case describes how a new user registers in the system.

Interaction Flow:

1. The user enters registration details in the user interface.
2. The Presentation Layer sends a request to the Auth Controller.
3. The Controller forwards the data to the Service Layer.
4. The Service Layer validates the input details.
5. The Repository Layer stores user data in the MySQL database.
6. The system sends a confirmation response to the user interface.

U2 – User Login

Actors: User

Description:
This use case describes how a registered user logs into the system.

Interaction Flow:

1. The user enters login credentials in the interface.
2. The Presentation Layer sends a request to the Auth Controller.
3. The Controller forwards the request to the Service Layer.
4. The Service Layer validates credentials.
5. The Repository Layer checks user data in the database.
6. The system grants access or displays an error message.

U3 – Submit Complaint

Actors: User

Description:
This use case describes how a user submits a complaint.

Interaction Flow:

1. The user fills the complaint form in the interface.
2. The Presentation Layer sends data to the Complaint Controller.
3. The Controller forwards the request to the Service Layer.
4. The Service Layer validates complaint details.
5. The Repository Layer stores complaint data in the database.
6. The system generates a complaint ID.
7. The system sends confirmation to the user interface.

U4 – Upload Supporting Files

Actors: User

Description:
This use case describes how a user uploads supporting files.

Interaction Flow:

1. The user selects a file to upload.
2. The Presentation Layer sends file data to the Complaint Controller.
3. The Controller forwards it to the Service Layer.
4. The Service Layer validates file type and size.
5. The Repository Layer stores file metadata in the database.
6. The system confirms successful upload.

U5 – View Complaint History

Actors: User

Description:
This use case describes how a user views past complaints.

Interaction Flow:

1. The user navigates to the complaint history page.
2. The Presentation Layer sends request to Complaint Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer retrieves complaint data.
5. The Repository Layer fetches data from database.
6. The system displays complaint history to the user.

U6 – Track Complaint Status

Actors: User

Description:
This use case describes how a user tracks complaint status.

Interaction Flow:

1. The user selects a complaint to track.
2. The Presentation Layer sends request to Complaint Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer retrieves status details.
5. The Repository Layer accesses database records.
6. The system displays status and updates.

U7 – Admin Login

Actors: Admin

Description:
This use case describes how an administrator logs into the system.

Interaction Flow:

1. The admin enters login credentials.
2. The Presentation Layer sends request to Admin Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer validates admin credentials.
5. The Repository Layer checks database records.
6. The system grants admin access or shows error.

U8 – View All Complaints

Actors: Admin

Description:
This use case describes how the admin views all complaints.

Interaction Flow:

1. The admin accesses the complaint dashboard.
2. The Presentation Layer sends request to Complaint Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer retrieves all complaint records.
5. The Repository Layer fetches data from database.
6. The system displays all complaints to the admin.

U9 – Filter Complaints

Actors: Admin

Description:
This use case describes how the admin filters complaints.

Interaction Flow:

1. The admin selects filter criteria.
2. The Presentation Layer sends request to Complaint Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer applies filtering logic.
5. The Repository Layer retrieves filtered data.
6. The system displays filtered complaints.

U10 – Update Complaint Status

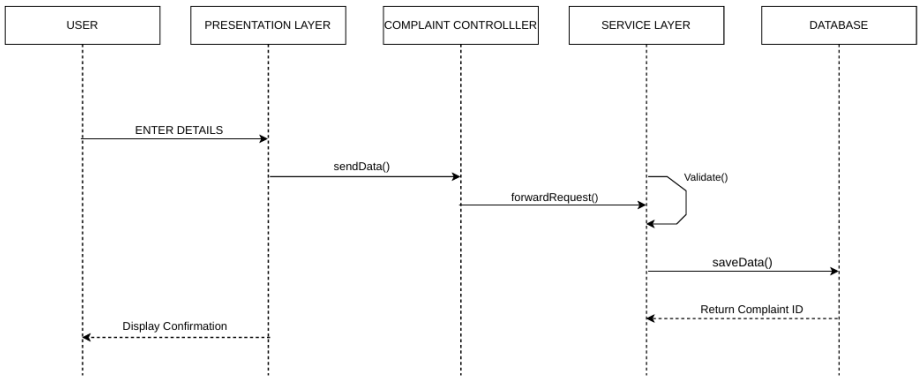
Actors: Admin

Description:
This use case describes how the admin updates complaint status.

Interaction Flow:

1. The admin selects a complaint.
2. The Presentation Layer sends update request to Admin Controller.
3. The Controller forwards request to Service Layer.
4. The Service Layer updates complaint status.
5. The Repository Layer saves changes in database.
6. The system confirms the update.

The following sequence diagram shows the interaction between objects for the complaint submission process.



5 Data View

Data View focuses on how data is organized, stored, and accessed within the Digital Complaint and Grievance Management System. The document describes all the key data entities in the system and their relationships, along with the structure of the database where that data is stored. This view is used to ensure data handling in the system is consistent, efficient, and reliable.

5.1 Domain Model

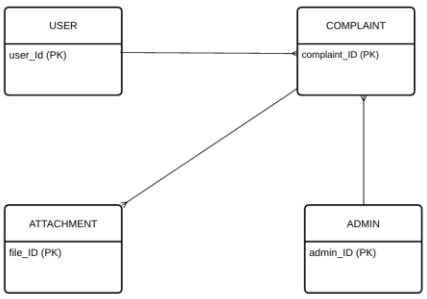
Entities:

- User
- Admin
- Complaint
- Attachment
- Response

Entity Relationships:

- A **User** can submit multiple **Complaints** (One-to-Many).
- Each **Complaint** belongs to one **User**.
- A **Complaint** can have multiple **Attachments** (One-to-Many).
- A **Complaint** can have responses from the **Admin**.
- An **Admin** manages multiple **Complaints**.

The following entity-relationship diagram shows the logical data structure.



5.2 Data Model (persistent data view)

Tables and Attributes:

1. Users Table

- user_id (Primary Key)
- name
- email
- password

2. Admin Table

- admin_id (Primary Key)
- username
- password

3. Complaint Table

- complaint_id (Primary Key)
- user_id (Foreign Key)
- title
- description
- status
- submission_date

4. Attachment Table

- file_id (Primary Key)
- complaint_id (Foreign Key)
- file_path

5. Response Table

- response_id (Primary Key)
- complaint_id (Foreign Key)
- message
- response_date

Relationships in Database:

- Users → Complaint (One-to-Many using user_id)
- Complaint → Attachment (One-to-Many using complaint_id)
- Complaint → Response (One-to-Many using complaint_id)

5.2.1 Data Dictionary

Data Element	Description	Type	Example
User_ID	Unique ID for user	Integer	1001
User_Name	Name of user	String	Ritu
Email	User email	String	user@gmail.com
Password	Encrypted password	String	******
Complaint_ID	Unique complaint ID	Integer	5001
Complaint_Title	Title of complaint	String	Network Issue
Complaint_Description	Detailed complaint	Text	Internet not working
Complaint_Status	Status of complaint	String	Pending
Submission_Date	Date of submission	Date	10-03-2026
Admin_Response	Admin reply	Text	Issue will be resolved
Attachment_File	Uploaded file	File	image.png

6 Exception Handling

Types of Exceptions and Handling

1. Authentication Exceptions

- **Condition:** Invalid username or password during login
- **Handling:**
  - System displays an error message such as “Invalid credentials”
  - User is prompted to re-enter login details

2. Validation Exceptions

- **Condition:** Missing or incorrect input (e.g., empty fields, invalid email format)

- **Handling:**
    - System highlights invalid fields
    - User is prompted to correct the input before submission

3. Database Exceptions

- **Condition:** Failure in database connection or data retrieval
- **Handling:**
    - System displays a generic error message
    - Error is logged for debugging
    - User is asked to try again later

4. File Upload Exceptions

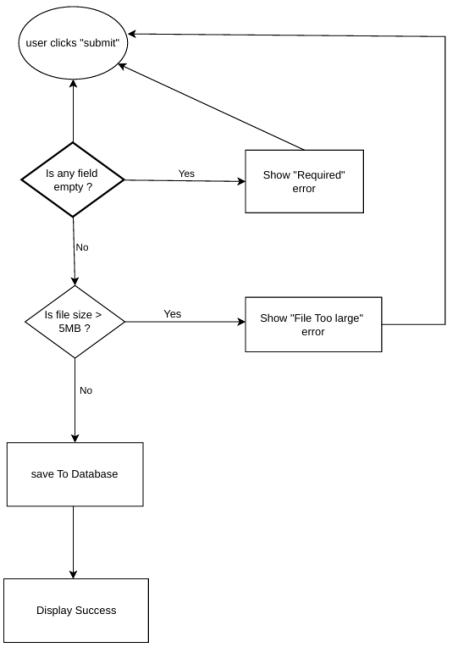
- **Condition:** Unsupported file format or file size exceeds limit
- **Handling:**
    - System rejects the file
    - Displays message indicating valid file requirements

5. Authorization Exceptions

- **Condition:** Unauthorized user tries to access admin features
- **Handling:**
    - Access is denied
    - User is redirected to login page

6. System Exceptions

- **Condition:** Unexpected server errors or API failures
- **Handling:**
    - System logs the error
    - Displays a user-friendly error message
    - Maintains system stability



7 Configurable Parameters

Configuration Parameter Name	Definition and Usage	Dynamic?
Max File Size	Defines maximum allowed file upload size	Yes
Allowed File Types	Specifies permitted file formats (e.g., jpg, png, pdf)	Yes
Session Timeout	Defines duration of user session	Yes
Database URL	Connection string for MySQL database	No
Server Port	Port on which backend server runs	No

8 Quality of Service

This section describes system characteristics related to availability, security, performance, and monitoring.

8.1 Availability

The system is designed to always be available to users under normal operating conditions. Maintenance activities may cause temporary downtime. Proper backend and database support ensures continuous operation.

8.2 Security and Authorization

The system ensures secure access through authentication and authorization mechanisms.

- Users must log in to access system features
- Passwords are stored securely (encrypted)
- Role-based access control is implemented
- Admin functionalities are restricted to authorized users

8.3 Load and Performance Implications

The system is designed to handle multiple users simultaneously.

- Response time is expected to be within three seconds under normal load
- Efficient database queries are used for faster retrieval
- Filtering and indexing improve performance

8.4 Monitoring and Control

The system maintains logs for monitoring and control purposes.

- User login activities are recorded
- Complaint updates are tracked
- Errors and system events are logged
- Logs help in debugging and system maintenance